CS 435: Cloud Networking	Spring 2026
Machine Problem 3: Managing a Data Center SDN	
Lead TA: Shehab Sarar Ahmed	*Due: April 19, 2026*

1 Overview

In this assignment, you will develop various features of a simulated data center network that uses a software-defined network (SDN) architecture.

- In the first part, you will write code to gather usage data from the data plane.
- In the second and third parts, you will implement two different routing strategies and observe how improved routing enhances performance through visual analysis.

2 Setup

2.1 Virtual machine

For this assignment, you will work on the course virtual machine. This is the standard course VM that you have used for the other MPs, unless you are using an Apple Silicon-based machine.

For Mac users with Apple Silicon only, the VM version that you used for MP1 and MP2 does not work for this assignment. Please download the shared image and add it in UTM. For detailed import steps, refer to the instructions in Week 1 materials.

2.2 Running the code

1. Those using an x86 processor can download the code for MP3 from Coursera. By default, the code should be downloaded into the Downloads folder. For those using an ARM processor and the VM we provided, the MP3 code should already be located in the `Downloads/MP3` folder.
2. If you are using an ARM processor, there's no need to run the installation script— all required packages are already pre-installed in the VM. If you're on an x86 processor and haven't run the `./install.sh` script during the MP1/2 setup, please run it now to install the necessary packages for MP3.
3. To start the emulator, open two terminals in the VM. In one of the terminals, run ''`sudo pip install eventlet==0.30.2`'' to downgrade a Python package to a version that our python program uses.
4. In terminal 1, cd to `Downloads/MP3` (you might need to also cd into the code folder depending on the processor) and run `sudo ./mdc --vid`

5. In terminal 2, cd to `Downloads/MP3/minidc/controller` or `Downloads/MP3/code/minidc/controller`
   - Start the network controller, Ryu: `ryu-manager controller.py`
   - The default (naive) routing policy will be loaded automatically.
6. Now that Ryu has started, press <enter> in **terminal 1**.
   - A Chrome window will pop up. Press play to start the video.
   - Observe the video quality is poor and plays only in low quality. The video may even pause to buffer.
7. End the experiment:
   - In terminal 1, type exit<enter> (**do not Ctrl-c to exit**; this will interrupt the teardown process). If you accidentally Ctrl-c and interrupt the process, run `sudo mn -c`.
   - In terminal 2, press Ctrl-c to stop Ryu.

3 Video Streaming

Now that the VM is set up, you will learn how to use the environment that emulates the miniature datacenter. Our system uses the network emulator Mininet to run a software-defined network (SDN). To control the SDN, we'll use the controller Ryu. This software is already installed and configured in the VM.

The datacenter will use a topology consisting of core switches, edge switches, and hosts, as shown below. Switches s104 and s105 are core switches, which connect to edge switches s101, s102, and s103. Each edge switch connects to all core switches, as well as two hosts (e.g., two of h1–h6). The datacenter will have two tenants – one running iperf to simulate bulk transfers and another streaming video. The video server will adapt the video quality to the available bandwidth between the video server and client.

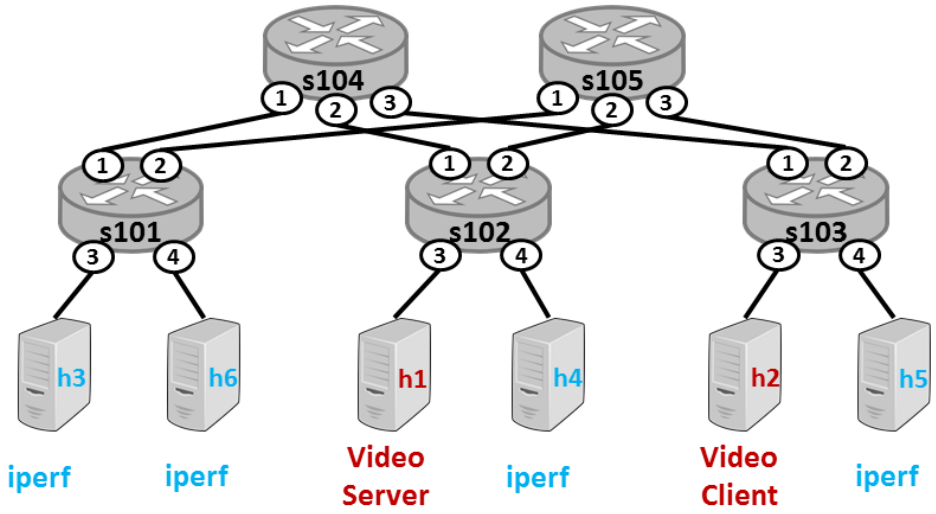

Figure 1: A sample topology of a data-center with six hosts from two virtual LANs, three edge switches and two core switches.

Each switch connects to another switch or host through a physical or virtual port. The physical ports are shown in the topology diagram as circled numbers. Each switch also has virtual ports to send packets to the SDN controller or flood traffic (i.e., send a packet out all physical ports except the one on which it was received).

You have already seen the impact of a poor routing policy on application performance across different tenants when you ran the code according to the guidelines in the previous section. What happened is that the traffic from the iperf tenants interfered with the available bandwidth to the video streaming tenant. This lack of performance isolation between tenants in a datacenter is not ideal. In different parts of this assignment, you will write policies that achieve better routing so applications meet their network performance needs.

The naive policy you saw in action routed all traffic from the edge switches to a single core switch, as is shown by the mixed red and blue lines. Notice that s104 can become a bottleneck in the network, and s105 is completely unused. As a result, the traffic of one tenant can easily impact another tenant's perceived network performance.

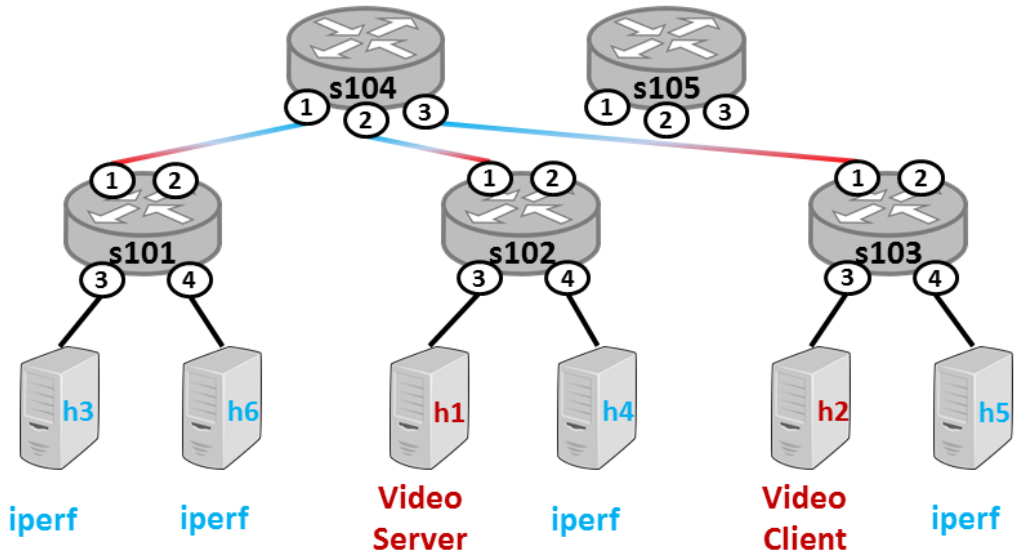

Figure 2: A sample topology where all traffic is routed through one single core switch.

4 Bandwidth Monitoring

Now that you have seen the impact of poor routing policies on application performance, your next step as network operator will be to quantify the problem by implementing a bandwidth monitor. To do this, you will extend our Ryu controller application to collect bandwidth information from the switches. Ryu and our emulated switches (like real data-center switches) support port statistics requests. This allows the controller application to query switches about the number of bytes, packets, and errors for each port (physical or virtual) on the switch.

We will be using the same topology and tenant placement as shown in Figure 1. In this part of the assignment, you will add code to the file `./minidc/controller/bwmon.py`. This file contains the code to issue a port statistics request to each switch in the network. Each switch responds asynchronously with a packet containing the statistics for each of its ports. Remember, each port is connected to a device on the network, such as a host or another switch.

All the code for issuing port statistics requests is provided for you. You will add a small amount of code to handle the reply. In particular, you will extend the function `statsReply Handler()` in the file `bwmon.py`. This function is invoked each time the controller receives a port statistics reply from a switch. The reply contains a list of statistics for each port. Your code should determine which host is associated with the port in the reply and update the bandwidth usage for that host. Useful API functions are detailed in the source file and comments. In particular, you can refer to OFPortStats in the Useful API functions to get the

port statistics. Your solution should not require any code outside the function `statsReply Handler()`. Hint: you only want to examine entries in the statistics list for physical ports, not virtual ports. You can filter out virtual ports by checking if the port has an entry in the list `self.topo.ports[switchName].keys()` for a switch switchName.

To understand the APIs and debug your code, you could print the variables (e.g. `print(self.topo.ports)`) and see how they are structured.

Note that you don't have to filter out the statistics of the core switches, as only the edge switches are connecting to hosts.

If coded correctly, you should see bandwidth statistics reported to the dashboard at `http://127.0.0.1:8080`. (Note: this page is only accessible when the script `mdc` is running. Also, opening a new window from the video player using Ctrl-N will open a sandboxed instance that does not have access to the dashboard. You will need to open the browser separately.) You will also see bandwidth usage statistics (grouped by tenant) printed to the terminal executing Ryu.

Expected Result If your code is working properly, you should see bandwidth data for each host and tenant in the dashboard at `http://127.0.0.1:8080`.

Submission Part 1 Submit the file `bwmon.py` containing your changes, in the "Programming Assignment 3 Part 1 Submission" item on Coursera. Remember: your solution should be contained within the function `statsReplyHandler()`.

5 Improving Routing

In the previous part, we learned how much each host contributes to the amount of congestion in the network. Now, you will improve the routing policy to provide better isolation between the tenants.

Till now, the routing policy sent all traffic from edge switches to only one core switch. You will improve the policy by distributing the traffic from edge switches across all core switches. Each core switch will have a set of one or more tenants assigned to it. Edge switches will then route traffic to the appropriate core switch based on which tenant the source and destination are associated with.

An example workflow is shown in Figure 3. There are two tenants (iperf and Video Server) and two core switches. s104 is associated with the Video Server tenant and s105 is associated with the iperf tenant. Hence, all packets from iperf and Video Server are sent to s105 and s104, respectively.

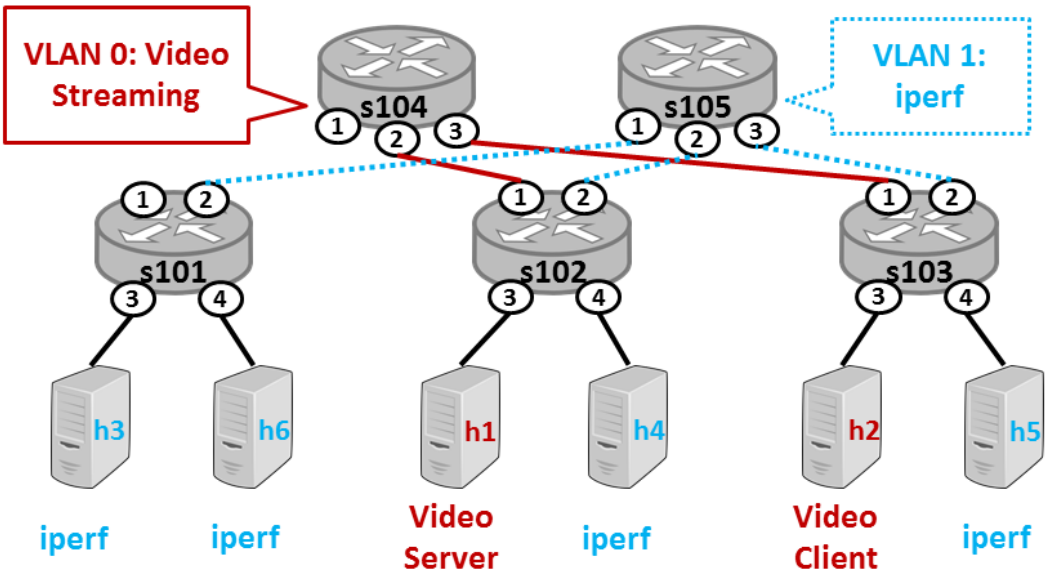

Figure 3: A sample topology where each core switch is responsible for a specific set of tenants, i.e., packets from these tenants are routed via that core switch.

Next, you will add code to the file `./minidc/controller/policy.py`. Specifically, you will extend the function `build()` in the class `StaticRouting`. Your code should install a rule in each edge switch. If the destination host is a neighbor – that is, if the host is directly connected to the switch via a port – output the packet out that port. If not, send the packet "up" to the core switch that is associated with the destination host's VLAN. You may use the "upward" rules in `DefaultPolicy.build()` as a rough guide. Useful API functions are also detailed in the source file and comments.

You can load your policy by first running the simulator according to the guidelines provided in Section 2.2, then opening a new, different instance of Chrome from the menu bar or Desktop, and finally selecting the radio button labeled "Static" and pressing the "Update Policy" button.

Expected Result If coded correctly, you should see an improvement in the video quality. The video may not play entirely in high quality, but you should notice an improvement compared to the default policy that routes all packets to one single core switch.

Submission Part 2 Submit the file `policy.py` containing your changes in the "Programming Assignment 3 Part 2 Submission" item on Coursera.

6 Optimized Routing

While the routing policy in the previous assignment utilizes all core switches, statically assigning core switches to route traffic for each VLAN can be problematic. Not all tenants use the same amount of bandwidth, so the load on the core switches may not be evenly distributed. As a result, one core switch may handle traffic for many tenants using large amounts of bandwidth while the other core switches are under-utilized. We will observe this scenario and update our routing policy to account for different traffic demands between tenants.

To illustrate this assignment, we use a larger topology shown in Figure 4, with 3 core switches, 4 edge switches, and 5 hosts per edge switch. There will be 3 different types of tenants, each with 4 hosts: Iperf, Inactive, and Memcached. The 2 Iperf tenants consist of hosts running iperf to simulate bulk transfers. Hosts that are members of the 2 Inactive tenants will not transfer any data. The Memcached tenant will consist of one client retrieving memcached objects, in parallel, from 3 different memcached servers. This will simulate a client loading a web page of multiple objects.

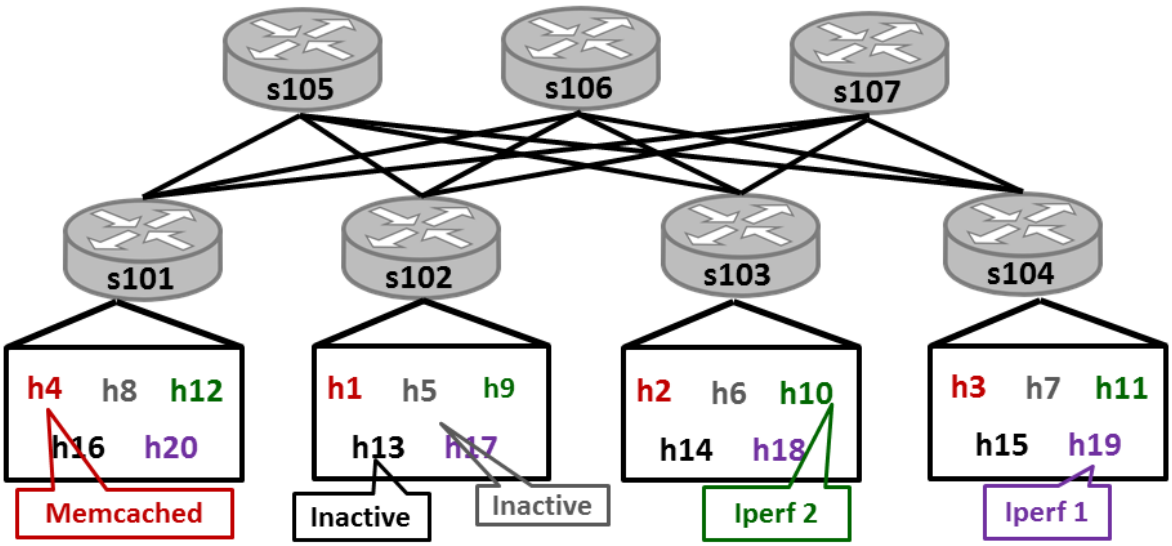

Figure 4: A sample topology with five tenants of three different types, each having four hosts each of which is connected with a different edge switch.

Rather than pre-assigning VLANs to particular core switches, your new routing policy will create a flow scheduling policy that balances flows across core switches, regardless of their VLAN/tenant association. You will implement this improved policy in the file `./minidc/controller/policy.py`. Specifically, you will add code to the function `minUtilization()` in the class `AdaptivePolicy`. Your code should use the dictionary `self.utilization` to find the least utilized core switch. This dictionary stores switch

names as keys and utilization (in bytes) as the value. Remember: since we are balancing the utilization among core switches, the dictionary `self.utilization` will only contain keys for core switches. Your code should not need to reference any other modules or objects aside from `self.utilization`.

We will explore this scenario using memcached, a caching system commonly used for caching objects in a web page. A client will issue requests to memcached servers and record the response time to load all the objects. The client will perform 20 page loads and record the median and 95th percentile from these 20 trials. The results will be reported at the bottom of the dashboard at `http://127.0.0.1:8080`.

To validate your code, run the simulator as stated in Section 2.2 **except in terminal 1, instead of** `sudo ./mdc --vid`, **use the command** `sudo ./mdc --adp`. Similar to the previous part, load your newly written policy by opening a new Chrome browser, and selecting the "Adaptive Policy" radio button. If coded correctly, your newest routing policy should significantly decrease the response time.

Expected Result If your code is working properly, you should see the memcached response time improve when the adaptive policy is loaded. In particular, you should see a noticeable improvement in the 95th percentile compared to the default and static policies.

Submission Part 3 Submit the file `policy.py` containing your changes in the "Programming Assignment 3 Part 3 Submission" item on Coursera. **Note:** Do not hard-code any values (e.g. name of the edge switch, host name) based on the topology mentioned in the MP spec. The auto-grader will create its own (mystery!) topology and the number (core/edge) switches and hosts are subject to change.

7 Team Formation

Although it is not required, we highly encourage you to complete MP3 in a two-student team. If you choose to work in a group, each student is expected to fully understand the entire codebase and be able to modify any part of it if asked during an interview or code review. Grading will be identical for individual and team submissions; there are no bonus points for working alone. Also, regardless of whether you work in a team or individually, each student must submit the assignment separately from their own account.

8 Grading Policy and Oral Interview

This MP will be graded through a combination of autograding of your code submission on Coursera (66% of the points), plus an oral interview (34% of the points). This interview will be individual. Each student must attend the oral interview alone, even if the MP was completed as part of a team. The interview will ask several questions to assess your

understanding of the your code, the concepts you used in the code, and the concepts in the MP assignment. It will not test whether your code is correct – only whether you understand and can explain it. Note that all team members are expected to understand all code submitted, regardless of which part each individual wrote. The interview will occur after the submission deadline; we will release details on how to sign up for a time slot later.

If you wrote the code yourself, then the interview should be straightforward. In the case that your interview performance indicates little to no understanding of your code (for example, receiving a score close to zero), we will assume you did not complete the work independently and that this constitutes an academic integrity violation. In that case, you may receive a score of 0 for both the autograded and interview portions of the MP.

9 Generative AI Usage Policy

In this MP, you may use generative AI tools to assist you – for example, to help understand the provided template code, learn relevant theoretical concepts, debug your code, or improve code that you have written yourself.

However, you are expected to design and implement the majority of your code yourself. You may not use AI tools to generate all of your solution or the main functionality in your solution.

Regardless of any AI assistance used, you must fully understand your implementation and be able to clearly explain all design decisions and code during evaluation. Oral interviews will be conducted to assess your understanding of the submitted MP.